End-to-End Risk Assessment of a Software System using MITRE ATT&CK, Vulnerability Analysis, STRIDE, and Risk Exposure

Learning objectives:

By the end of this project, students will be able to:

- Model multi-stage cyber attacks using the MITRE ATT&CK Framework
- Develop and analyze vulnerable software
- Perform vulnerability assessment and interpret CVSS scores
- Apply STRIDE threat modeling
- Quantify risk using Exposure Factor and ALE
- Propose and evaluate security controls
- Understand risk management across SDLC phases
-

You are working as a Risk Manager in a software development company responsible for securing a newly deployed web-based student management system. During routine monitoring, you suspect a multi-stage cyber attack targeting user credentials and sensitive academic data. To investigate, you simulate an attack using structured stages aligned with the MITRE ATT&CK Framework. You develop a Python-based prototype containing intentional vulnerabilities to replicate real-world weaknesses. Through vulnerability assessment, threat modeling, and risk exposure analysis, you identify critical risks. You then implement security controls, retest the system, and evaluate the effectiveness of mitigation strategies within the SDLC process.

Phase 1: Attack Scenario Design (Planning Phase)
Task:

- Design a multi-stage attack scenario (refer Annexure 1)
- Use at least 7 MITRE tactics

Requirements:

- Clearly define:
    - Attack goal (e.g., data breach, privilege escalation)
    - Attack path (step-by-step)

Deliverable:

- Attack flow diagram or table
- Map vulnerabilities (with CVSS scores) to MITRE stages in an attack pathway

Example Tactics:

- Initial Access
- Execution
- Persistence
- Privilege Escalation
- Defense Evasion
- Collection
- Exfiltration
- Impact

Phase 2: Vulnerable System Development (Development Phase)

Task:
Develop a Python-based application (e.g.):

- Login system
- Student management system
- File upload system

<u>Note: You can make your own functionality.</u>

Requirements:
Include intentional vulnerabilities, such as:

- **Refer Annexure 1**

Phase 3: Vulnerability Assessment (Testing Phase)
Tools (suggested):

- Bandit (Python security scanner)
- OWASP ZAP (optional)

Task:

- Scan your code
- Identify vulnerabilities
- Assign CVSS scores

Deliverable:

Vulnerability	Description	CVSS Score	Severity

Phase 4: CI/CD Integration (Optional but Recommended)
Tools:

- Jenkins / GitHub Actions

Task:

- Integrate code scanning into pipeline
- Automate:
  - Code analysis
  - Security checks

Phase 5: Threat Modeling using STRIDE
Task:
Deliverable:
Map threats to your application components

Phase 6: Risk Quantification
Task:
For at least 2 risks, calculate:

- Asset Value (AV)
- Exposure Factor (EF)
- Annual Rate of Occurrence (ARO)

Formula:
ALE=AV×EF×AROALE = AV \times EF \times AROALE=AV×EF×ARO

Deliverable:
| Risk | AV | EF | ARO | ALE |

Phase 7: Control Design & Code Review (Implementation Phase)
Task:

- Fix vulnerabilities
- Implement controls:
  - Input validation
  - Secure authentication
  - Logging

Phase 8: Retesting & Control Effectiveness
Task:

- Re-run vulnerability scan
- Compare before vs after

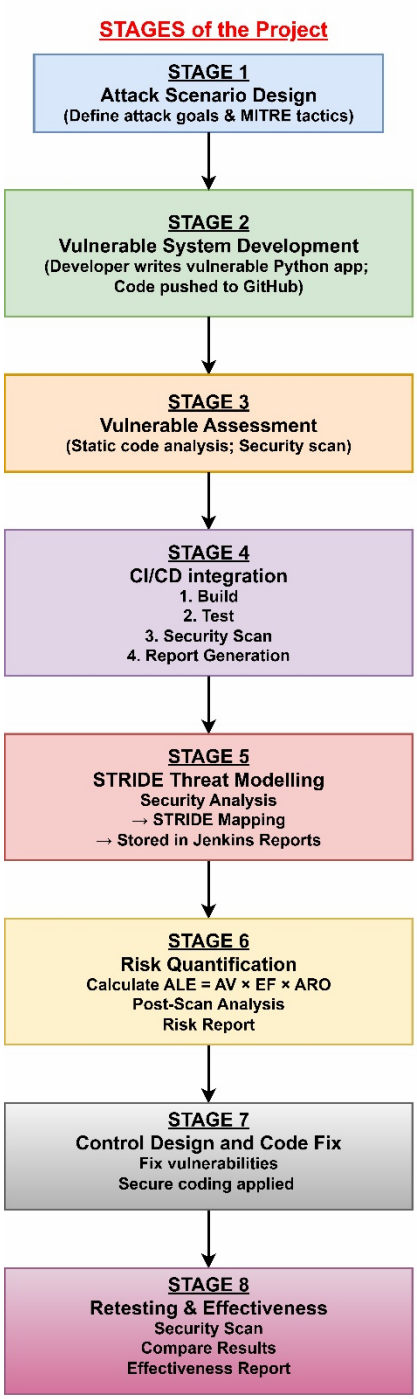
Deliverable:

Vulnerability	Before	After	Control Applied	Effectiveness

Final Deliverables
Students must submit:

1. Report (PDF)
2. Source Code (before & after fixes)
3. Risk Register
4. Vulnerability Assessment Report
5. STRIDE Analysis Table
6. Risk Exposure Calculations
7. Control Effectiveness Table

Summarized Process Flow.



Deliverable: (Group of 4 members)

- Report that will include the following
  - Project description, Stage of RM and its calculation, Scanning using Bandit, CD/CI Pipeline Jenskins.
  - Dataset:
    - Code file of the vulnerability.
    - Patched codes file of the vulnerability.

Annexure 1 (Possible list of Attacks for reference)

Category	**Attack Type**	**Description**	**Possible Impact**	**STRIDE Mapping**
Credential-Based	Brute Force	Repeated login attempts	Unauthorized access	Spoofing
Credential-Based	Credential Stuffing	Use of leaked passwords	Account takeover	Spoofing
Credential-Based	Session Hijacking	Stealing session cookies	Unauthorized session access	Spoofing
Injection	SQL Injection	Modify/extract database data	Data breach, data manipulation	Tampering, Information Disclosure
Injection	Command Injection	Execute OS commands	System compromise	Tampering, Elevation of Privilege
File-Based	Insecure File Upload	Upload malicious scripts	Remote code execution	Tampering, Elevation of Privilege
File-Based	Path Traversal	Access restricted files	Sensitive file exposure	Information Disclosure
Auth & Access Control	Broken Authentication	Weak passwords/no lockout	Unauthorized login	Spoofing
Auth & Access Control	Privilege Escalation	User becomes admin	Full system control	Elevation of Privilege
Data Exposure	Sensitive Data Exposure	Plain-text/unsecured data	Confidential data leak	Information Disclosure
Data Exposure	Information Disclosure	Verbose error messages	System information leak	Information Disclosure
Application Logic	Parameter Tampering	Modify URL parameters	Unauthorized actions	Tampering

Application Logic	Business Logic Flaw	Bypass application rules	Unauthorized access to data	Tampering, Elevation of Privilege
Network-Based	Man-in-the-Middle	Intercept communication	Data theft	Information Disclosure, Spoofing
DoS	Login Flooding	Excessive login requests	System unavailability	Denial of Service
Logging Failure	Log Injection	Manipulate logs	Hide attacker activity	Repudiation
Logging Failure	No Logging	No audit trail	Actions cannot be traced	Repudiation

Tutorial for Mapping CVE with Mitre

Objective: CVE provides us with a single value score of a vulnerability which corresponds to the severity of vulnerabilities in other words the impact it will have on the system. The scale is 1-10. As an analyst we can look for a probable attack pathway by the description of the vulnerability. So there is a need for Mapping both of it and the series of step will help you to do it.

Lets take an example of a CVE CVE-2021-44228, CVSS Score = 10 (Critical)

Affected Software Apache Log4j

Vulnerability Description

A critical remote code execution (RCE) vulnerability in Log4j allows attackers to execute arbitrary code by sending specially crafted input that gets logged by the application.

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

Meaning:

- Remote attack (Network)
- No authentication required
- Full system compromise possible

Step 1: Map CVE to MITRE Attack Stages

Stage	**MITRE Tactic**	**Technique**	**Role of CVE-2021-44228**

Step 1	Initial Access	Exploit Public-Facing App	Malicious input triggers vulnerability
Step 2	Execution	Command Execution	Attacker executes remote code
Step 3	Persistence	Create Account / Backdoor	Attacker maintains access
Step 4	Privilege Escalation	Exploit for Privilege	Gains higher access
Step 5	Collection	Data from System	Access sensitive data
Step 6	Exfiltration	Data Transfer	Sends data outside
Step 7	Impact	System Manipulation	Data deletion / ransomware

Step 2: Understand CVSS in the Attack Chain

Exploit Log4j → Remote Code Execution → Install Backdoor → Steal Data → Exfiltrate → Damage System

“Why is this CVSS score 10.0?”

Expected reasoning:

- No authentication required
- Easy to exploit
- Full system takeover

MITRE + CVSS + Stage Importance Mapping

MITRE Stage	**Technique**	**Vulnerability (Example)**	**CVSS**	**Stage Importance**	**Stage Risk (CVSS × Importance)**	**Why It Matters**
Initial Access	Valid Accounts	Weak Password Policy	5.3	4	21.2	Entry point enables attack chain
Execution	SQL Injection	Input Validation Failure	9.8	5	49.0	Core exploit, full system access

Privilege Escalation	Role Manipulation	Improper Access Control	7.5	4	30.0	Expands attacker privileges
Collection	Data Access	Unencrypted Database	7.0	3	21.0	Enables sensitive data retrieval
Exfiltration	Data Transfer	Lack of Network Monitoring	6.5	5	32.5	Final data loss occurs here

Sample Output.

Step 3: Bandit Scanning (Python based scanning tool, SAST)

Objective: It is a recursive scanning tool which will help the students to detect vulnerabilities in the python code.

Tutorial for Python Code Review

Sample Project

You are part of a software development team tasked with building a **secure, scalable web application** for a fictional organization that handles sensitive user data (e.g., user accounts, file uploads, and APIs). The organization has faced prior security incidents due to poor coding practices, lack of automated testing, and weak access controls.
Your objective is to perform end to end risk management pipeline in the context of software development.
Students must:

1. Develop a functional web application with core features:
    - User authentication (login/signup)
    - Role-based access (Admin/User)
    - At least one API endpoint
    - File upload or data input module
2. Apply **secure coding practices** to prevent:
    - SQL Injection
    - Cross-Site Scripting (XSS)
    - Broken authentication
    - Improper input validation

System Requirements

- **Functional Requirements**
  - User registration and authentication
  - Role-based access control (Admin/User)
  - Secure API endpoint
  - File upload or user input processing
- **Security Requirements**
  - Input validation and sanitization
  - Password hashing (e.g., bcrypt)
  - Secure session handling
  - Protection against OWASP Top 10 vulnerabilities

Technology Requirements (Recommended)

- **Backend:** Python (Flask) / Java (Spring Boot) / Node.js
- **Database:** MySQL / PostgreSQL

Security Tools:

- SAST → Bandit / Semgrep
- DAST → OWASP ZAP